# 第七节 压缩解压及其关键技术

# 课程安排

1. 软件工程与软件建模
2. 敏捷开发与项目协作
3. 开发环境规范与测试
4. AI辅助开发
5. 需求设计与程序框架
6. 打包解包及其关键技术
7. 压缩解压及其关键技术
8. 加密解密与系统集成
9. 软件发布与项目收尾
10. 项目答辩与成果展示

# 实验难度分级和评分标准

## 基本要求

各小组“独立”实现一款数据备份软件（对应**基础分**）：

数据备份：将目录树中的文件数据保存到指定位置

数据还原：将目录树中的文件数据恢复到指定位置

## 扩展要求

各项目组根据自身情况自行选择扩展要求（对应**扩展分**）。

**文件类型支持**：支持特定文件系统的特殊文件（管道/软链接/硬链接等）

**元数据支持**：支持特定文件系统的文件元数据（属主/时间/权限等）

**自定义备份**：允许用户筛选需要备份的文件（路径/类型/名字/时间/尺寸）

**打包解包**：将所有备份文件拼接为一个大文件保存

**压缩解压**：通过文件压缩节省备份文件的存储空间

**加密解密**：由用户指定密码，将所有备份文件均加密保存

**图形界面**：实现友好易用的GUI界面

**定时备份**：允许用户进行设置，进行周期性定时备份和数据淘汰

**实时备份**：自动感知用户文件变化，进行自动备份

**网络备份**：将数据备份软件从单机模式扩展为网盘模式，还涉及到的功能包括：用户管理、元数据管理、传输加密、增量备份等。

**其它功能**：视功能难度讨论加分。

# 压缩&解压

# 编码解码

在电文传输中，需要将电文中出现的每个字符进行二进制**编码**。在设计编码时需要遵守两个原则：
（1）发送方传输的二进制编码，到接收方解码后必须具有**唯一性**，即解码结果与发送方发送的电文完全一样；
（2）发送的二进制编码尽可能地**短**。

## 等长编码

每个字符的**编码长度相同**（编码长度就是每个编码所含的二进制位数）。这种编码的特点是解码简单且具有唯一性，但编码长度并不一定是最短的。

## 不等长编码

每个字符的**编码长度不同**。

压缩（无损）：若对出现频度较高的字符（字符串）分配相对较短的编码，同时，对出现频度较低的字符分配相对较长的编码，则编码之后的数据的二进制位数可以变小。

# 压缩编码分类

基于统计的方法：Huffman编码、Shannon-Fano编码

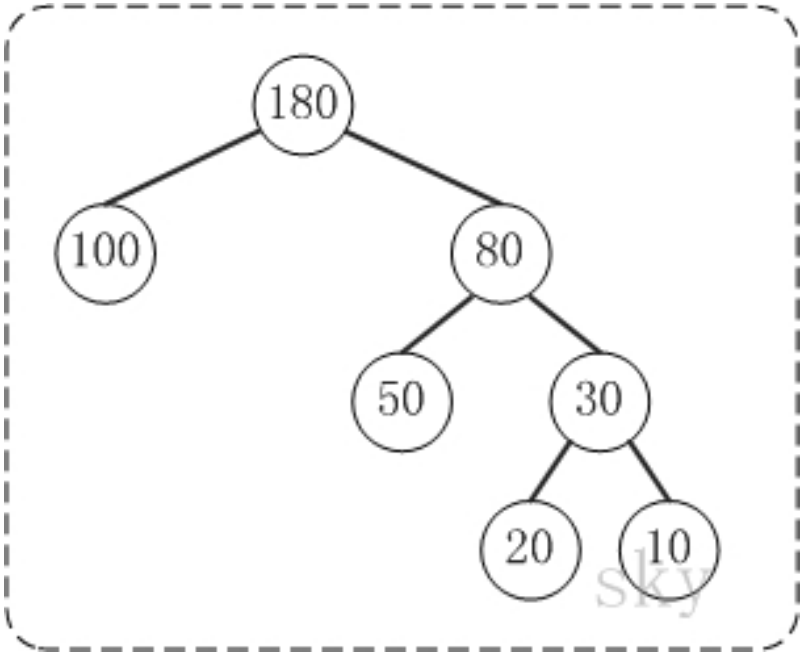
基于字典的方法：LZ77算法、 LZ78算法

混合方法：DEFLATE算法

以Huffman编码和LZ77算法为例。

# 哈夫曼树

定义：给定n个权值作为n个**叶子结点**，构造一棵二叉树，若树的**带权路径长度（WPL）**达到最小，则这棵树被称为哈夫曼树。




**例子**：示例中，

WPL

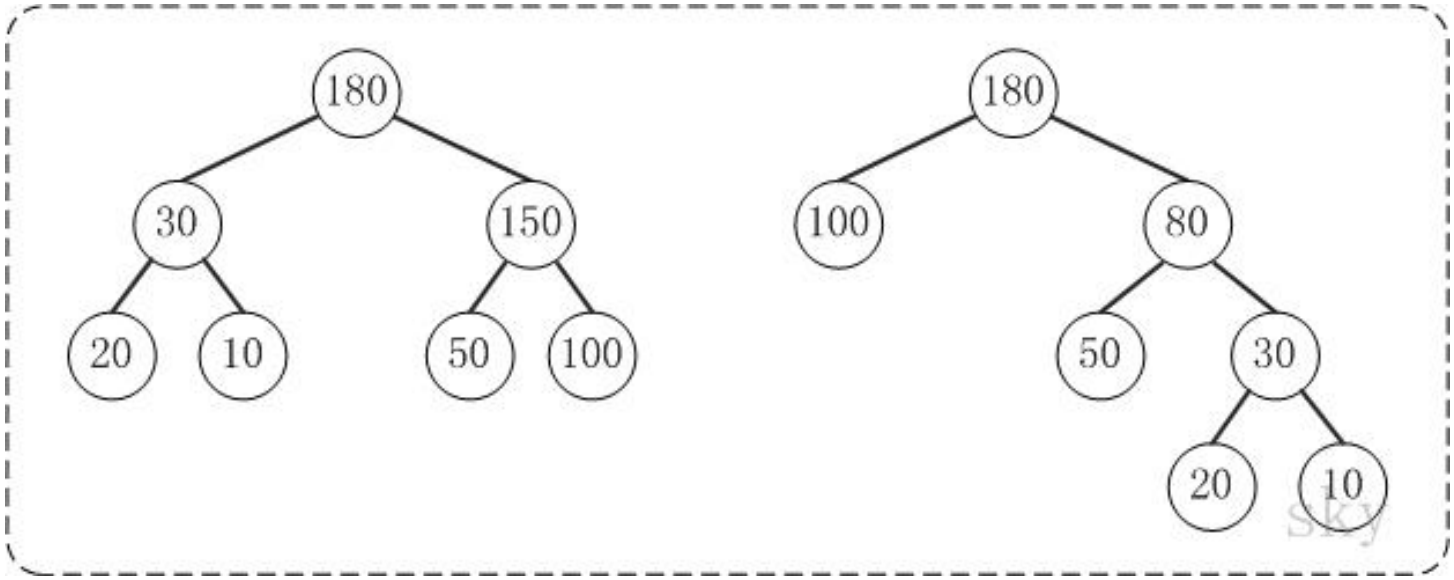
= 1*100 + 2*50 + 3*20 + 3*10

= 100 + 100 + 60 + 30

= 290

# 例子

比较下面两棵树：




左边的树WPL= 2*10 + 2*20 + 2*50 + 2*100 = 360
右边的树WPL= 1*100 + 2*50 + 3*20 + 3*10 = 290

**问题：**基于带有权值的叶子结点$w_1$、$w_2$、…，$w_n$，怎样构建哈夫曼树？

# 构造哈夫曼树

**算法思路：**

（**1**）将$w_1$、$w_2$、…，$w_n$看成是有n 棵树的森林(每棵树仅有一个结点)；

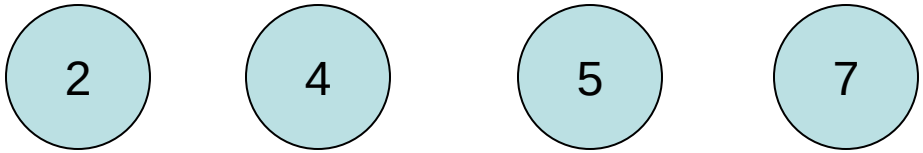
（**2**）在森林中选出根结点的**权值最小的两棵树进行合并**，作为一棵新树的左、 右子树，且新树的根结点权值为其左、右子树根结点权值之和；

（**3**）从森林中删除选取的两棵树，并将新树加入森林；

（**4**）重复(02)、(03)步，**直到森林中只剩一棵树为止**，该树即为所求得的哈夫曼树。

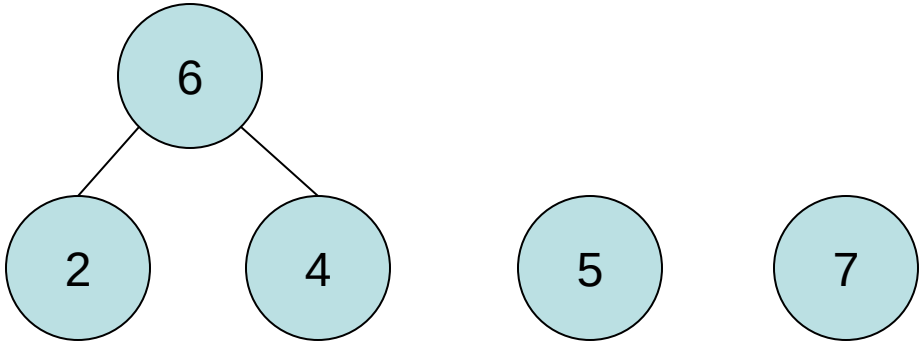
# 构造哈夫曼树示例

当权值为{7，5，2，4}时，构造哈夫曼树。



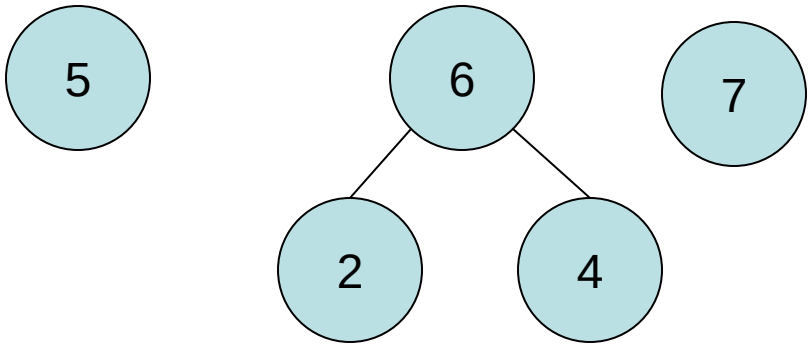
# 构造哈夫曼树示例

当权值为{7，5，2，4}时，构造哈夫曼树。



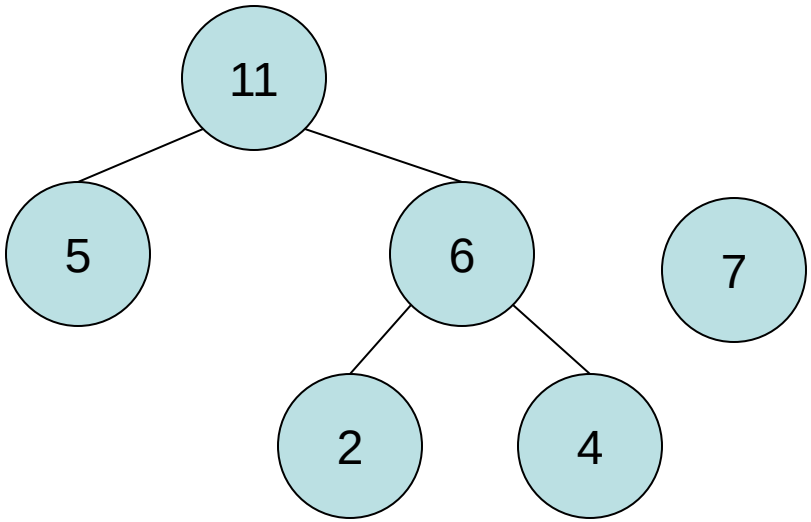
# 构造哈夫曼树示例

当权值为{7，5，2，4}时，构造哈夫曼树。



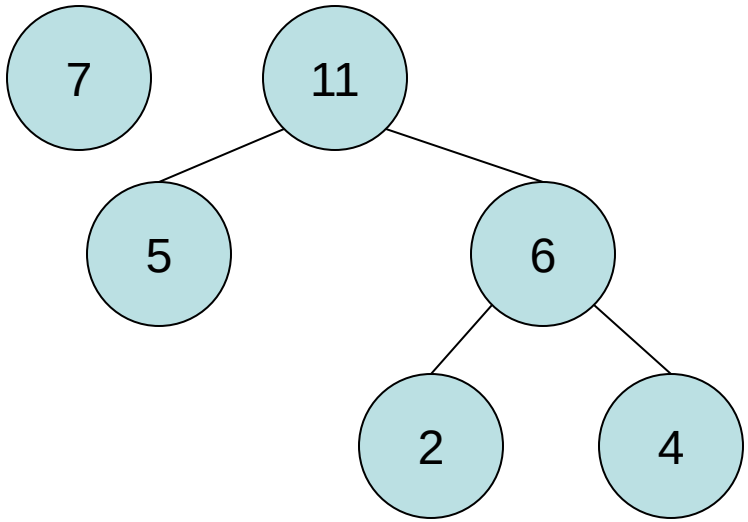
# 构造哈夫曼树示例

当权值为{7，5，2，4}时，构造哈夫曼树。



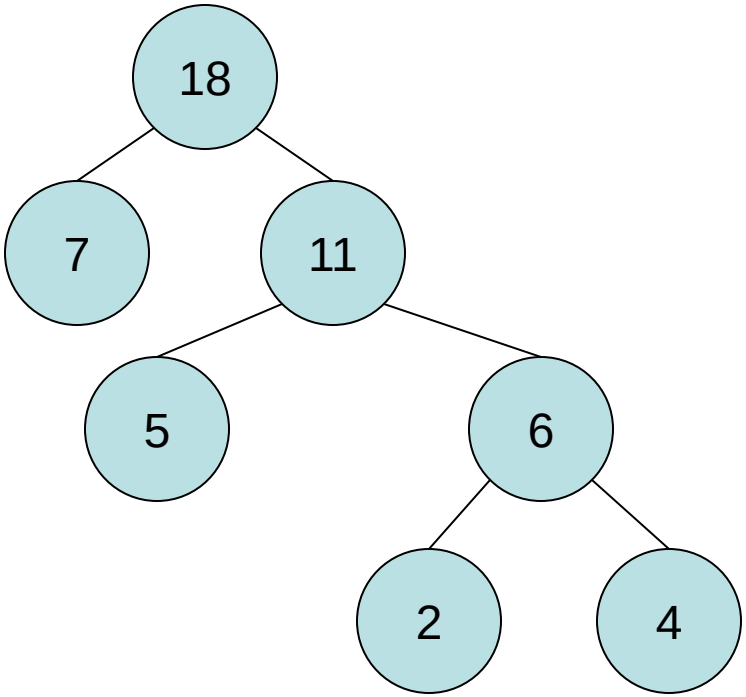
# 构造哈夫曼树示例

当权值为{7，5，2，4}时，构造哈夫曼树。



# 构造哈夫曼树示例

当权值为{7，5，2，4}时，构造哈夫曼树。



# 哈夫曼编码

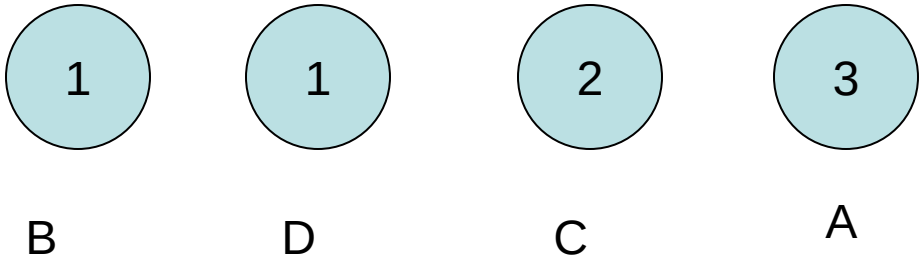
**算法思路：**

（1）利用字符集中每个字符（叶子结点）的使用频率作为权值构造哈夫曼树。

（2）从根结点开始，为其每个子结点赋予不同的**但顺序相同的**编码（如：左分支赋0，右分支赋1）。将从根结点到该叶子结点的路径编码作为该字符的编码。

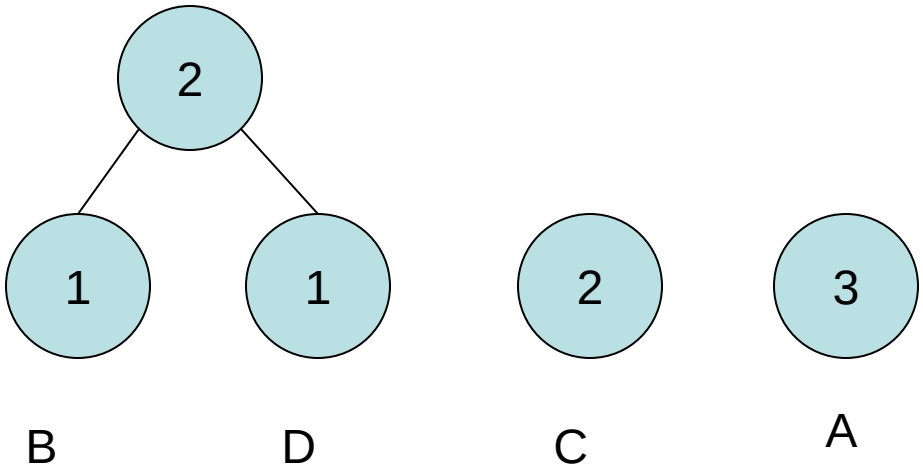
# 哈夫曼编码举例：ABACCDA

对于字符串“ABACCDA”，共有7个字符，4种字符。其中A、B、C、D出现的次数分别为3、1、2、1。根据权值{3，1，2，1} 构造哈夫曼树



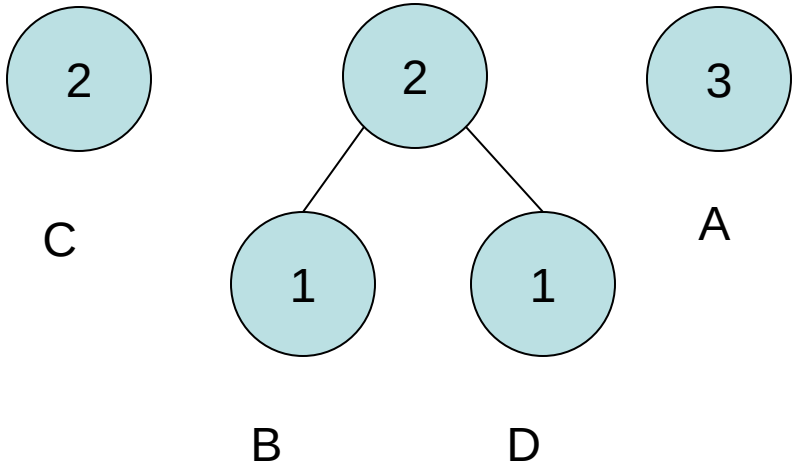
# 哈夫曼编码举例：ABACCDA

对于字符串“ABACCDA”，共有7个字符，4种字符。其中A、B、C、D出现的次数分别为3、1、2、1。根据权值{3，1，2，1} 构造哈夫曼树



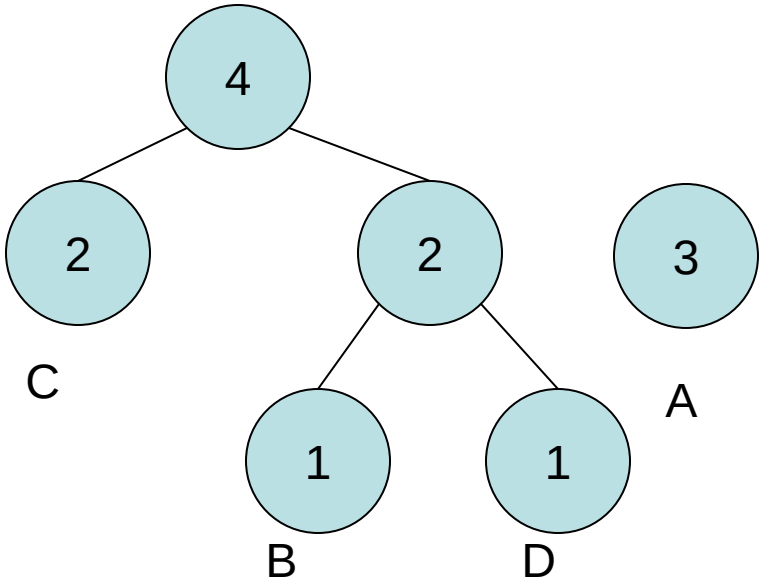
# 哈夫曼编码举例：ABACCDA

对于字符串“ABACCDA”，共有7个字符，4种字符。其中A、B、C、D出现的次数分别为3、1、2、1。根据权值{3，1，2，1} 构造哈夫曼树



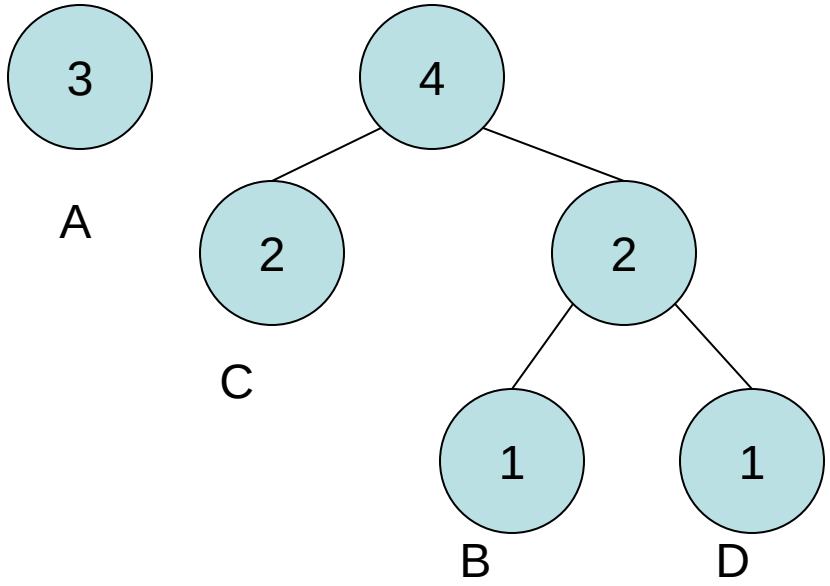
# 哈夫曼编码举例：ABACCDA

对于字符串“ABACCDA”，共有7个字符，4种字符。其中A、B、C、D出现的次数分别为3、1、2、1。根据权值{3，1，2，1} 构造哈夫曼树



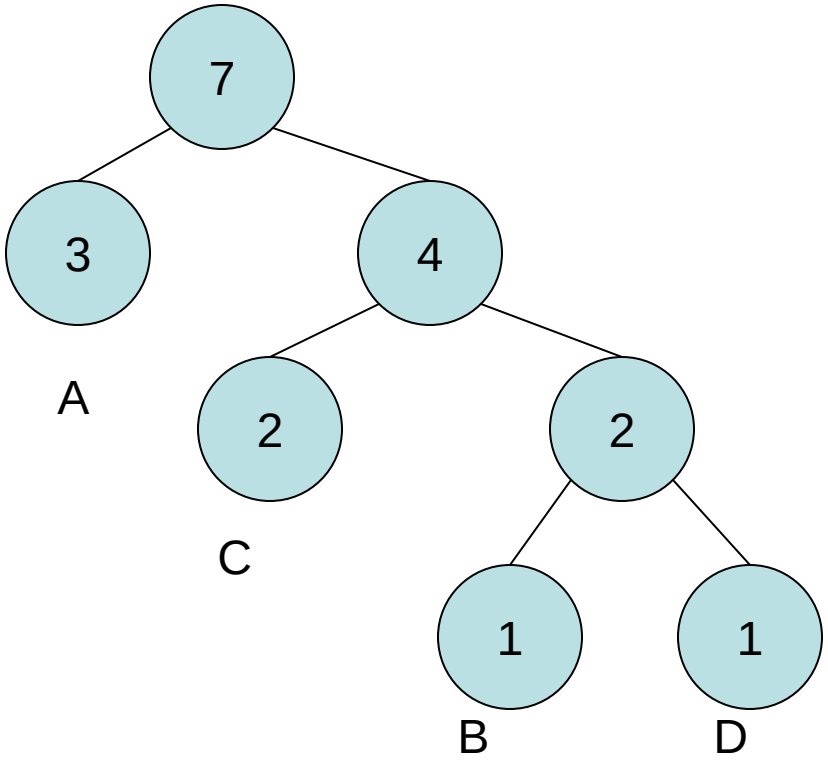
# 哈夫曼编码举例：ABACCDA

对于字符串“ABACCDA”，共有7个字符，4种字符。其中A、B、C、D出现的次数分别为3、1、2、1。根据权值{3，1，2，1} 构造哈夫曼树



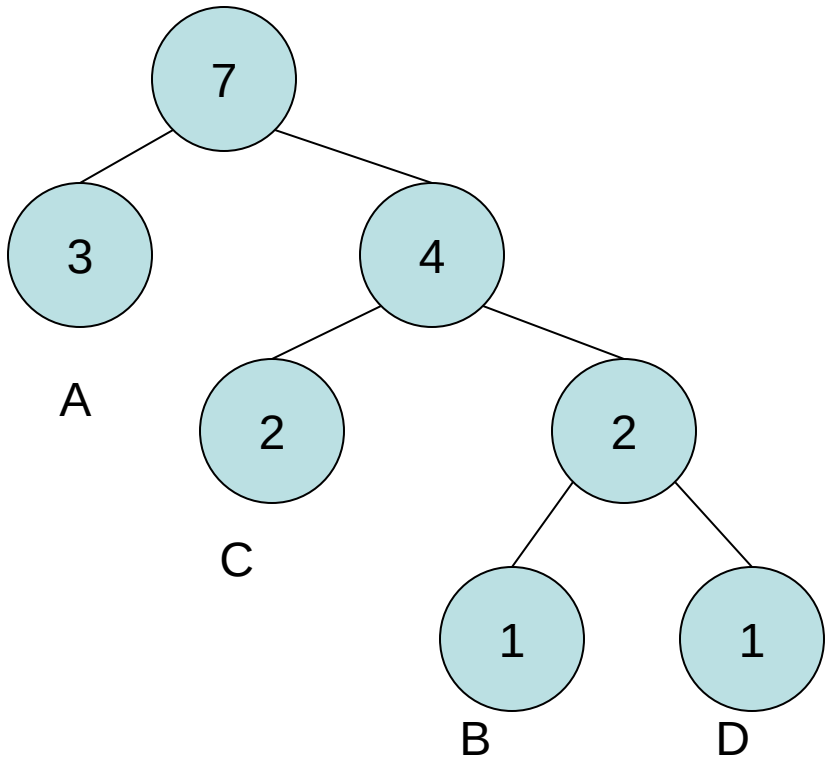
# 哈夫曼编码举例：ABACCDA

对于字符串“ABACCDA”，共有7个字符，4种字符。其中A、B、C、D出现的次数分别为3、1、2、1。根据权值{3，1，2，1} 构造哈夫曼树



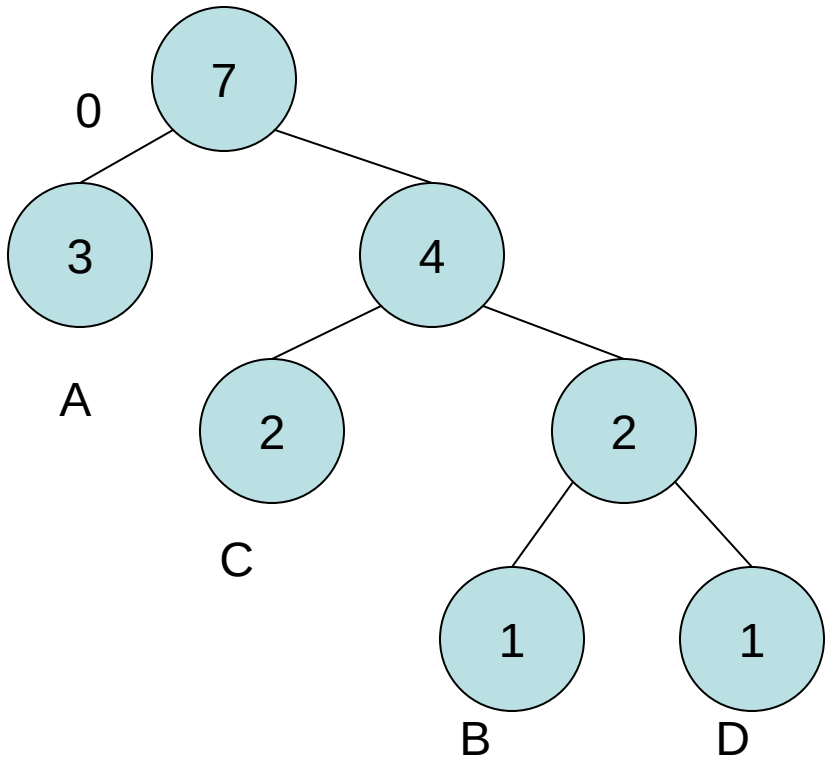
# 哈夫曼编码举例：ABACCDA

对哈夫曼树进行编码，每一个结点的左分支上面标0，右分支上面标1。



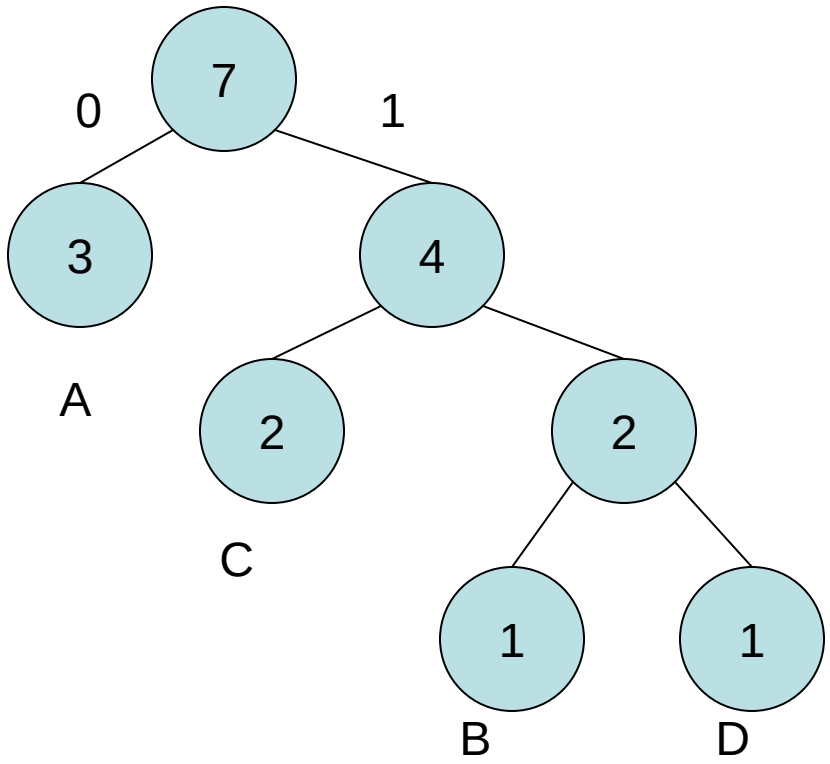
# 哈夫曼编码举例：ABACCDA

对哈夫曼树进行编码，每一个结点的左分支上面标0，右分支上面标1。



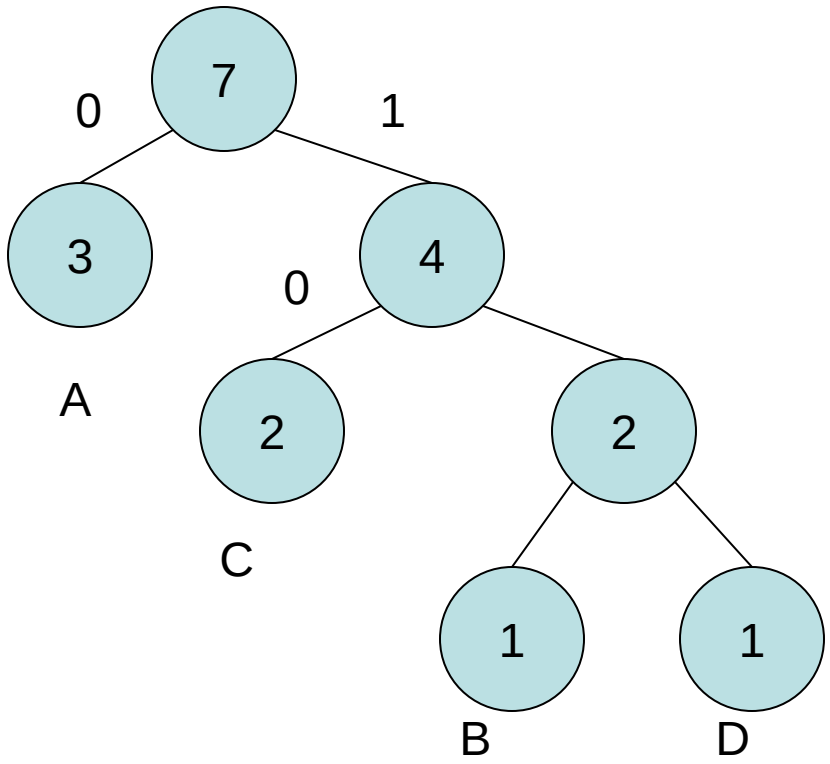
# 哈夫曼编码举例：ABACCDA

对哈夫曼树进行编码，每一个结点的左分支上面标0，右分支上面标1。



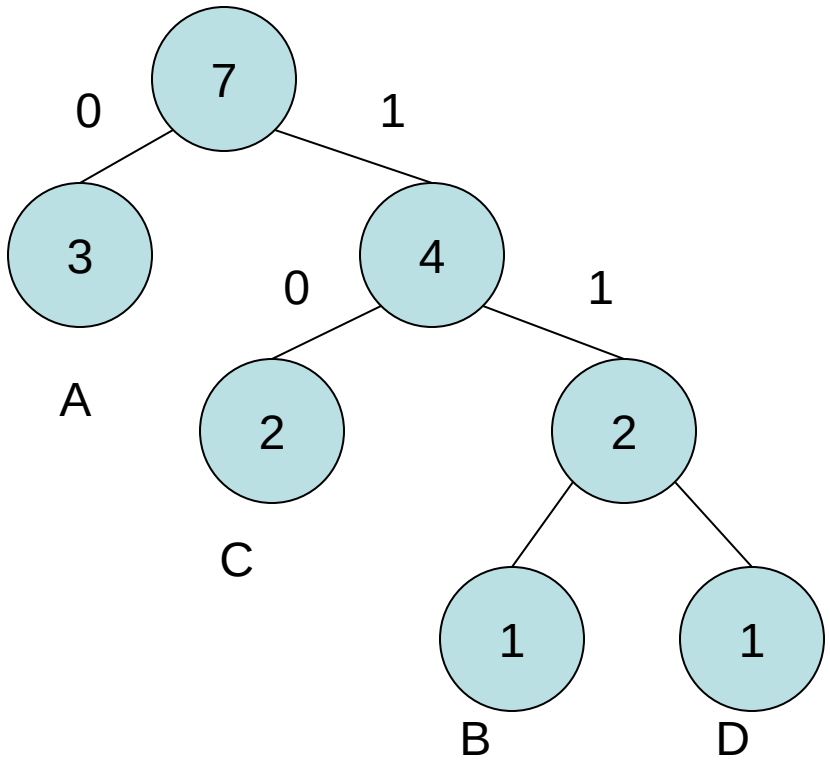
# 哈夫曼编码举例：ABACCDA

对哈夫曼树进行编码，每一个结点的左分支上面标0，右分支上面标1。



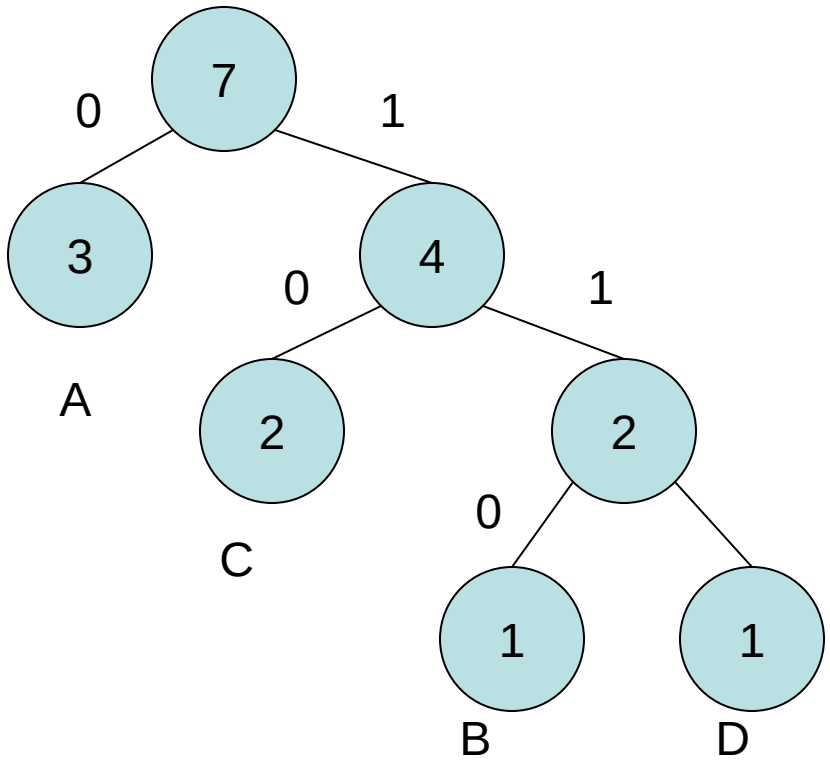
# 哈夫曼编码举例：ABACCDA

对哈夫曼树进行编码，每一个结点的左分支上面标0，右分支上面标1。



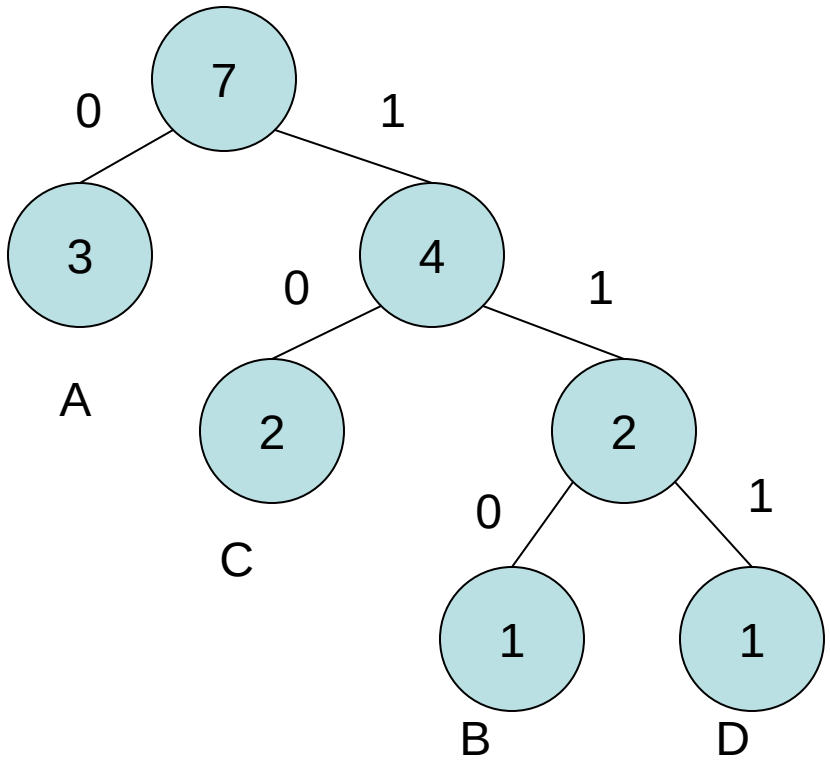
# 哈夫曼编码举例：ABACCDA

对哈夫曼树进行编码，每一个结点的左分支上面标0，右分支上面标1。



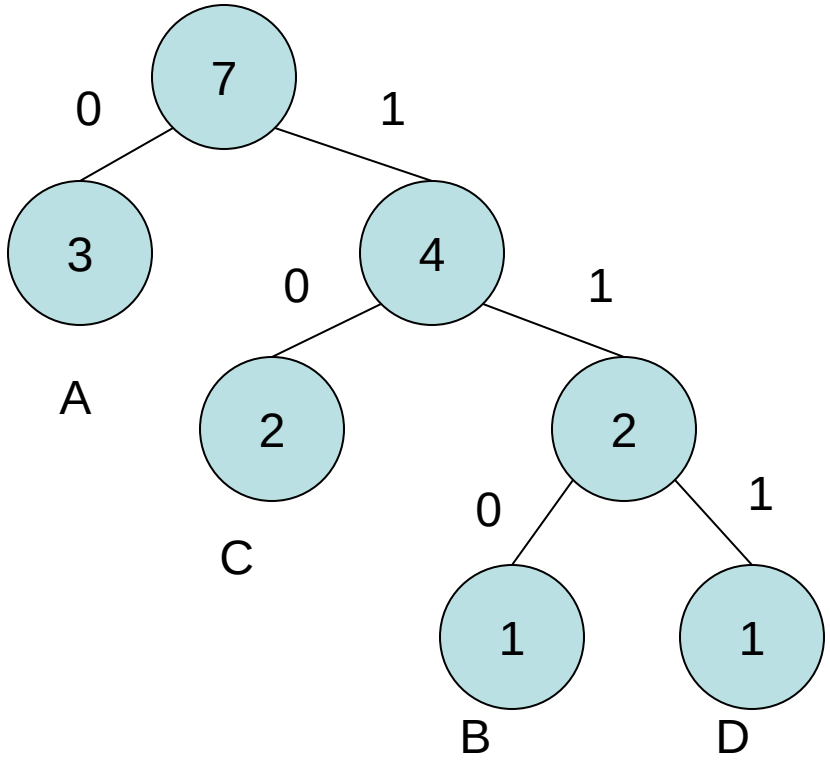
# 哈夫曼编码举例：ABACCDA

对哈夫曼树进行编码，每一个结点的左分支上面标0，右分支上面标1。



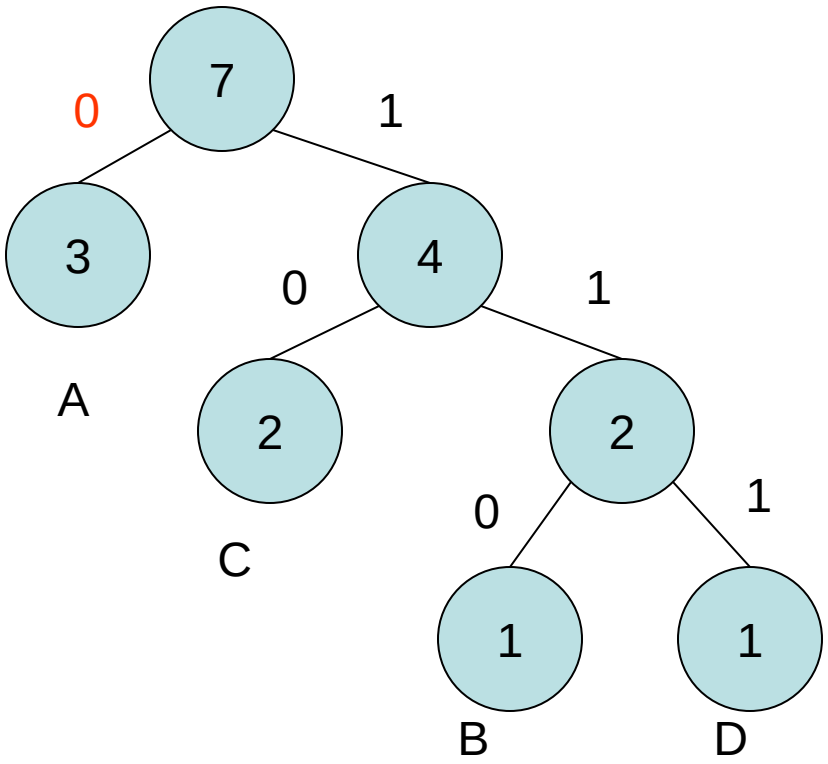
# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。



# 哈夫曼编码举例：ABACCDA

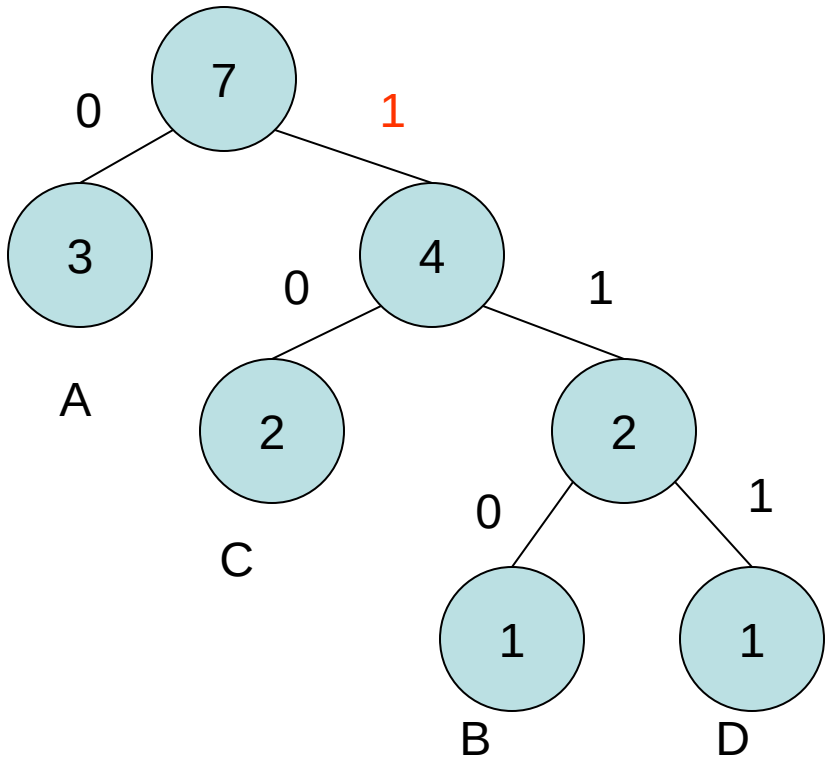
从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。




A：0

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。

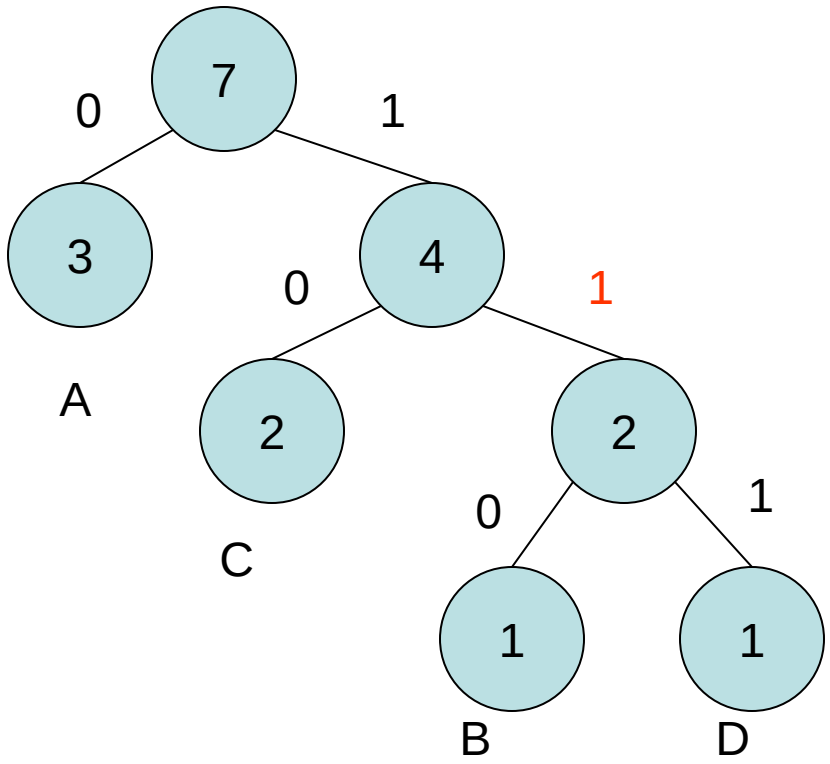



A：0

B：1

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。

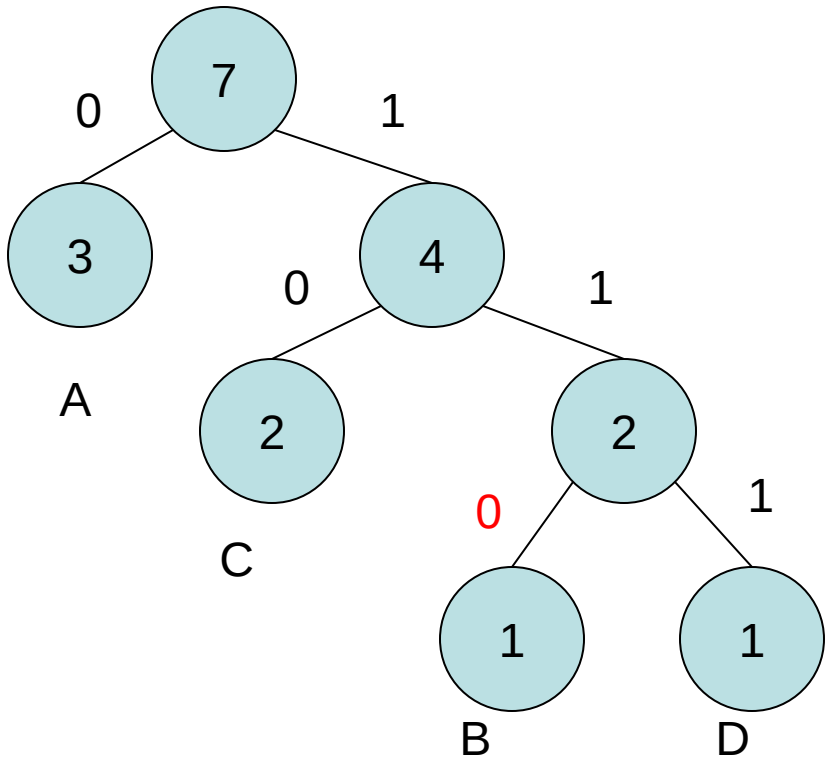



A：0

B：11

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。

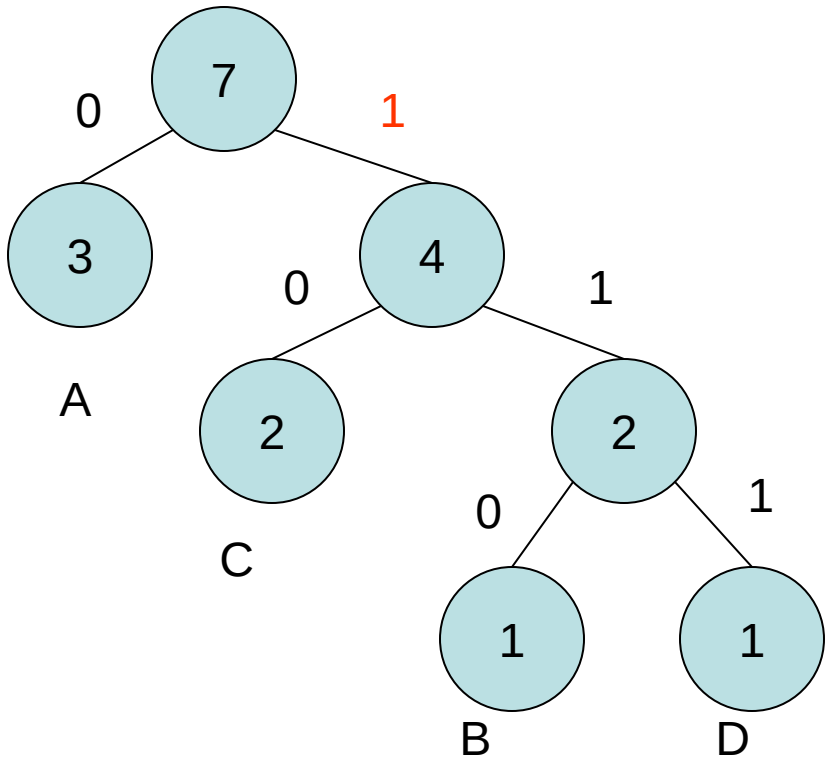



A：0

B：110

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。



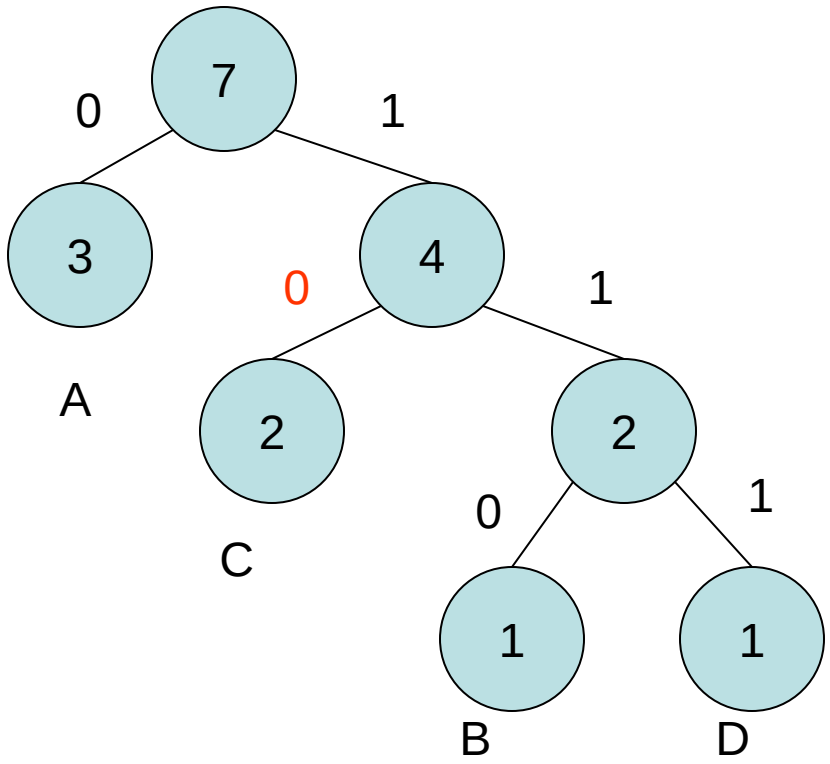

A：0

B：110

C：1

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。



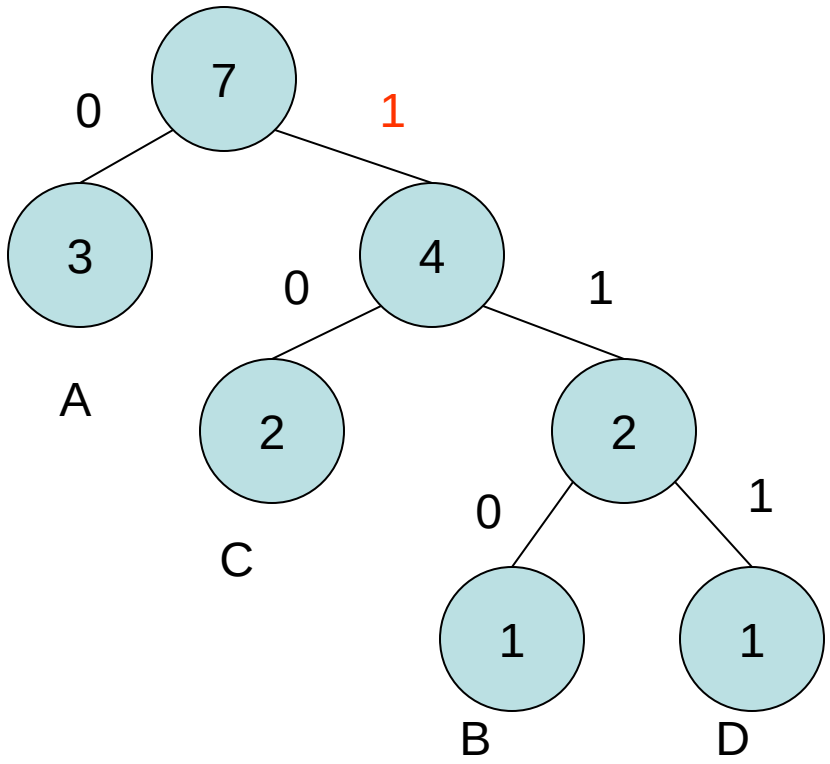

A：0

B：110

C：10

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。




A：0

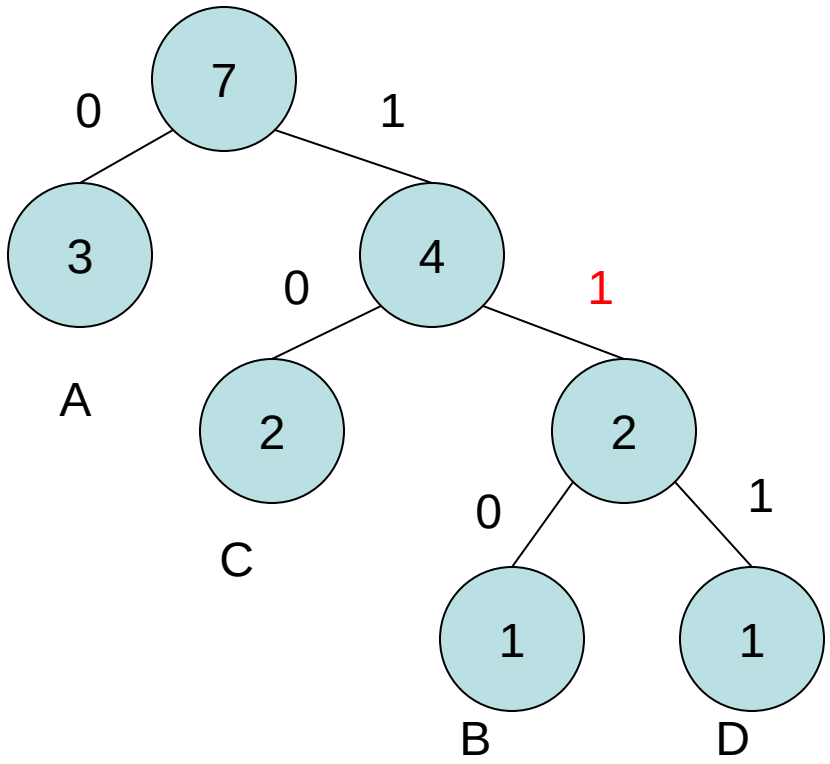
B：110

C：10

D：1

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。




A：0

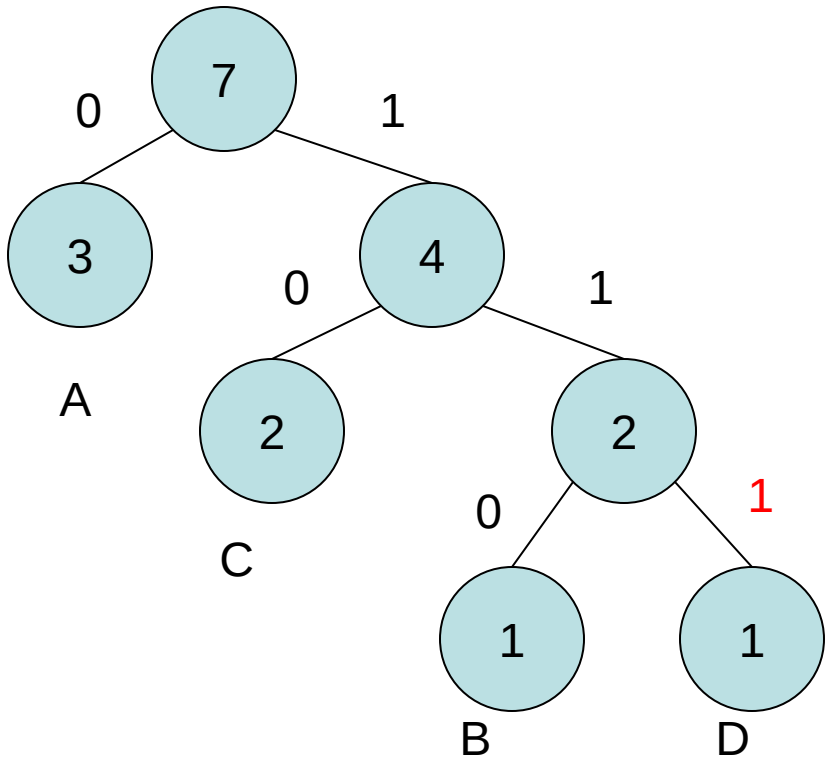
B：110

C：10

D：11

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。




A：0

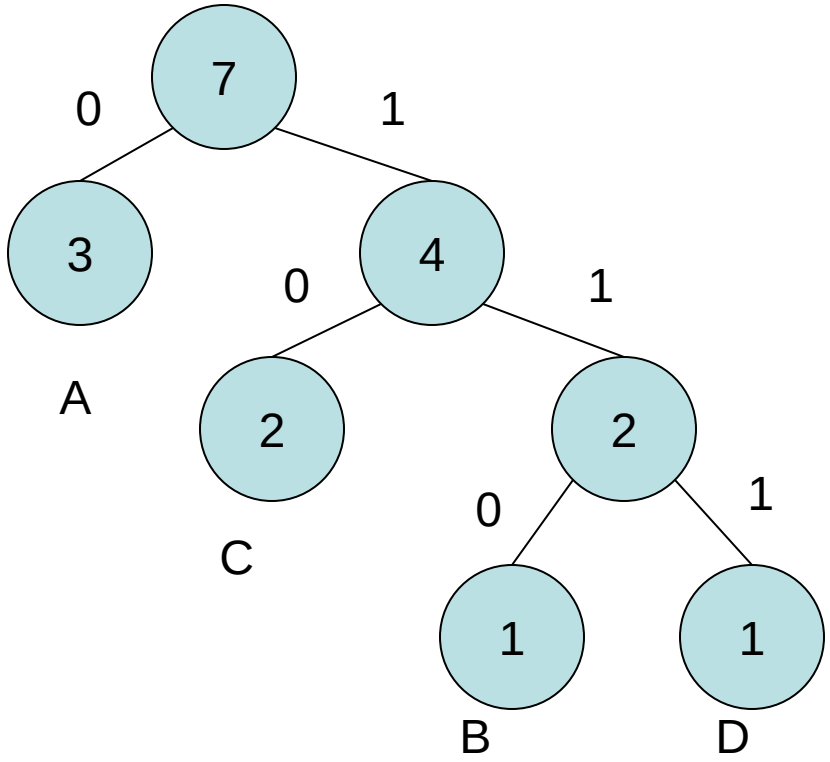
B：110

C：10

D：111

# 哈夫曼编码举例：ABACCDA

从根结点到各个叶子结点，所经分支上面的0、1序列，就是该叶子结点所代表字符的编码。




A：0

B：110

C：10

D：111

ABACCDA

0110010101110

# 使用哈夫曼编码进行文件压缩

目标

思路

基于单词编码进行文件压缩

# 使用哈夫曼编码进行文件压缩

目标

基于单词编码进行文件压缩

思路

统计词频

# 使用哈夫曼编码进行文件压缩

**目标**

基于单词编码进行文件压缩

**思路**

统计词频

⇩

根据字母出现次数？
根据汉字出现次数？

# 使用哈夫曼编码进行文件压缩

目标

基于单词编码进行文件压缩

思路

统计词频

⇩

~~根据字母出现次数？~~
~~根据汉字出现次数？~~

⇩

文件在内存都是二进制存储的
根据Byte出现次数

# 使用哈夫曼编码进行文件压缩

**目标**

基于单词编码进行文件压缩

**思路**

统计词频

⇩

~~根据字母出现次数？~~
~~根据汉字出现次数？~~

⇩

文件在内存都是二进制存储的
根据Byte出现次数

对文件以二进制方式读入，然后对每8个bit，即1个Byte（刚好对应256个ACSII字符）进行统计，这样就可以统计出文件对应的256个字符的权值。

# 使用哈夫曼编码进行文件压缩

目标

基于单词编码进行文件压缩

思路

统计词频



~~根据字母出现次数？~~
~~根据汉字出现次数？~~



文件在内存都是二进制存储的
根据Byte出现次数



根据Byte词频构建哈夫曼树

对文件以二进制方式读入，然后对每8个bit，即1个Byte（刚好对应256个ACSII字符）进行统计，这样就可以统计出文件对应的256个字符的权值。

# 使用哈夫曼编码进行文件压缩

目标

基于单词编码进行文件压缩

⇧

获得每个单词的编码

⇧

根据Byte词频构建哈夫曼树

思路

统计词频

⇩

~~根据字母出现次数？~~
~~根据汉字出现次数？~~

⇩

文件在内存都是二进制存储的
根据Byte出现次数

⇦

对文件以二进制方式读入，然后对每8个bit，即1个Byte（刚好对应256个ACSII字符）进行统计，这样就可以统计出文件对应的256个字符的权值。

# 使用哈夫曼编码进行文件压缩

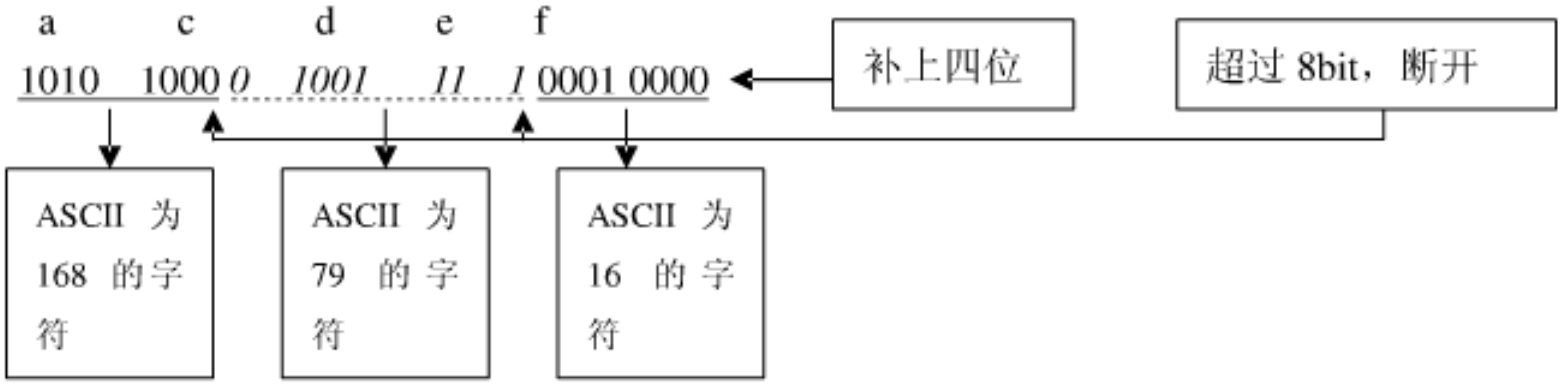
**注意：不要用char数据类型来保存每一位编码。**这样得到的编码实际上是由字符'0'和字符'1'组成的字符串，不但不能使文件得到压缩，反而会使文件增大。

**解决方案：位操作！**

举例如下：

已知：a: 1010　b: 00　c: 10000　d: 1001　e: 11　f: 10001　g: 01　h: 1001

如果其中的一段是:: acdef 则处理如下：




**注意：**需要记录真实编码位数或者补位的位数。否则，最后一个Byte无法正确解码。

# 使用哈夫曼编码进行文件压缩

**需要存储的用于解码的信息：**

1、编码之后的数据

2、编码后数据的真实长度（或补位的长度）

3、解码用数据

# 解码用数据存储方法

方法一：编码对照表

A：0

B：110

C：10

D：111

编码对照表存储方式：

字符+编码位长度+编码（补零）

# 解码用数据

方法一：编码对照表

A：0

B：110

C：10

D：111

编码对照表存储方式：

字符+编码位长度+编码（补零）

方法二：词频表

A：3

B：1

C：2

D：1

直接记录原始文件中各字符出现的频率，以及构造哈夫曼树时的叶子顺序。
解码时读取该频率信息，并基于存储的叶子顺序，重新生成一棵同样的哈夫曼树。

# 解码用数据

方法一：编码对照表

A：0

B：110

C：10

D：111

方法二：词频表

A：3

B：1

C：2

D：1

编码对照表存储方式：

字符+编码位长度+编码（补零）

直接记录原始文件中各字符出现的频率，以及构造哈夫曼树时的叶子顺序。
解码时读取该频率信息，并基于存储的叶子顺序，重新生成一棵同样的哈夫曼树。

方法三：直接存储哈夫曼树

树的静态存储方式

# 解码用数据

方法一：编码对照表

A：0

B：110

C：10

D：111

编码对照表存储方式：

字符+编码位长度+编码（补零）

方法二：词频表

A：3

B：1

C：2

D：1

直接记录原始文件中各字符出现的频率，以及构造哈夫曼树时的叶子顺序。
解码时读取该频率信息，并基于存储的叶子顺序，重新生成一棵同样的哈夫曼树。

方法三：直接存储哈夫曼树

树的静态存储方式

**思考**：解码时最方便使用的解码用数据是什么？

# 工程实现

Huffman压缩5步曲

- 1. 统计词频（wc.table[word]++）
- 2. 写入词频表到目标文件
- 3. 构建哈夫曼树
- 4. 生成编码字典（dict）
- 5. 执行编码

词频统计：

```
#define WORD_NUM 256

struct WordCount {
    long int table[WORD_NUM];
};

while ((ret = sfFile.readN(&word, sizeof(uint8_t))) > 0) {
   wc.table[word]++;
}
```

按字节（0-255）进行统计：兼容任意二进制文件

# 工程实现

Huffman压缩5步曲

- 1. 统计词频（wc.table[word]++）
- 2. 写入词频表到目标文件
- 3. 构建哈夫曼树
- 4. 生成编码字典（dict）
- 5. 执行编码

词频统计：

```
#define WORD_NUM 256

struct WordCount {
    long int table[WORD_NUM];
};

while ((ret = sfFile.readN(&word, sizeof(uint8_t))) > 0) {
   wc.table[word]++;
}
```

按字节（0-255）进行统计：兼容任意二进制文件

# 工程实现

Huffman压缩5步曲

- 1. 统计词频（wc.table[word]++）
- 2. 写入词频表到目标文件
- 3. 构建哈夫曼树
- 4. 生成编码字典（dict）
- 5. 执行编码

写入词频表：

```
if (targetFile.writeN(&wc, sizeof(wc)) < 0) {
    std::cout << "写入词频表失败! " << std::endl;
    return -1;
}
```

压缩后的文件头部需存储词频表，否则不能解压！

# 工程实现

Huffman压缩5步曲之3：构建哈夫曼树

构造哈夫曼树：　手写二叉树？在**生产代码中不要重复造轮子！**

**哈夫曼树：优先取频率最低的节点构成树**

**priority_queue：优先级队列，大顶堆　人工指定比较器，让小的在堆顶**

```
struct NodeCompare {
    bool operator()(HuffmanNode *&a, HuffmanNode *&b) {
        if (a->freq > b->freq) {
            return true;    // true 表示 “a 的优先级低于 b”
        } else if (a->freq == b->freq) {
            return a->data > b->data;  // 用于构造严格序
        } else {
            return false;
        }
    }
};
```

std::priority_queue<T, Container, Compare> 中的 Compare 是一个**仿函数（functor）**，满足：**comp(a, b) == true 表示 a 应该排在 b 的后面（即 b 比 a 更优先）**。

# 工程实现

Huffman压缩5步曲之3：构建哈夫曼树

构造哈夫曼树：　手写二叉树？在**生产代码中不要重复造轮子！**

**哈夫曼树：优先取频率最低的节点构成树**

**构造小顶堆**

**节点数据类型　　容器底层数据结构　　大小比较规则**

```
std::priority_queue<HuffmanNode *, std::vector<HuffmanNode *>, NodeCompare> pq;

for (int i = 0; i < WORD_NUM; i++) {
    if (wc.table[i] > 0) {                    // 词频非零者入队，构造小顶堆
        pq.push(new HuffmanNode(static_cast<uint8_t>(i), wc.table[i]));
    }
}
```

# 工程实现

Huffman压缩5步曲之3：构建哈夫曼树　**从小顶堆中依次取值构造哈夫曼树**

```
while (pq.size() > 1) {
        HuffmanNode *left = pq.top(); // 取出频率最小的两个节点
        pq.pop();
        HuffmanNode *right = pq.top();
        pq.pop();

        HuffmanNode *parent =
            new HuffmanNode(left->data, /* 用于构造严格序 */ left->freq + right->freq);
        parent->left = left;
        parent->right = right;

        // 将父节点加入优先队列
        pq.push(parent);
    }

    // 返回根节点
    return pq.top();
```

# 工程实现

Huffman压缩5步曲之4：生成字典　　**递归的艺术**

```
void generateCodeDict(HuffmanNode *root, std::string code, std::vector<std::string> &dict) {
    if (root->left == NULL && root->right == NULL) {
        dict[root->data] = code;  // 叶子节点：存编码
    }
    generateCodeDict(root->left, code + "0", dict);
    generateCodeDict(root->right, code + "1", dict);
}
```

**先序遍历，左0右1，走到叶子就存编码**

# 工程实现

Huffman压缩5步曲之5：进行编码 **编码是“0101”字符串，但文件存二进制字节！**

```
BitVec::BitVec(const std::string &str) { // str长度必须为8
    value = 0;
    int shift = 7;
    for (char c : str) {
        if (c == '1') value |= (1u << shift);
        shift--;
    }
}
```

**哈夫曼编码后长度不一定是字节的整数倍**

```
if (codecBuffer.size() % 8 != 0) {
    paddingNum = 8 - (codecBuffer.size() % 8);
    for (...) codecBuffer += '0';
}
// ... 写入 paddingNum 到文件
```

**补0对齐，但必须把补了多少位记下来（paddingNum），否则解压时会多解出垃圾数据！**

# 工程实现

Huffman压缩5步曲之6：解压

不需要编码字典，只需要哈夫曼树！
从根开始，按比特走：0往左，1往右，走到叶子就是原始字节

```
ptr = root;
for (each bit) {
    if (bit=='0') ptr = ptr->left;
    else ptr = ptr->right;
    if (leaf) { output data; ptr = root; }
}
```

**为什么不用字典解压？”**
**因为树结构天然支持流式解码，内存更省，且避免存储额外映射表。**

# LZ77算法

基于统计的压缩编码算法（如Huffman编码）需要事先得到数据的出现频率。但有时（如流数据压缩），这种先验知识很难预先获得。因此，需要更为通用的压缩编码算法。

LZ77：基于滑动窗口的动态字典编码。

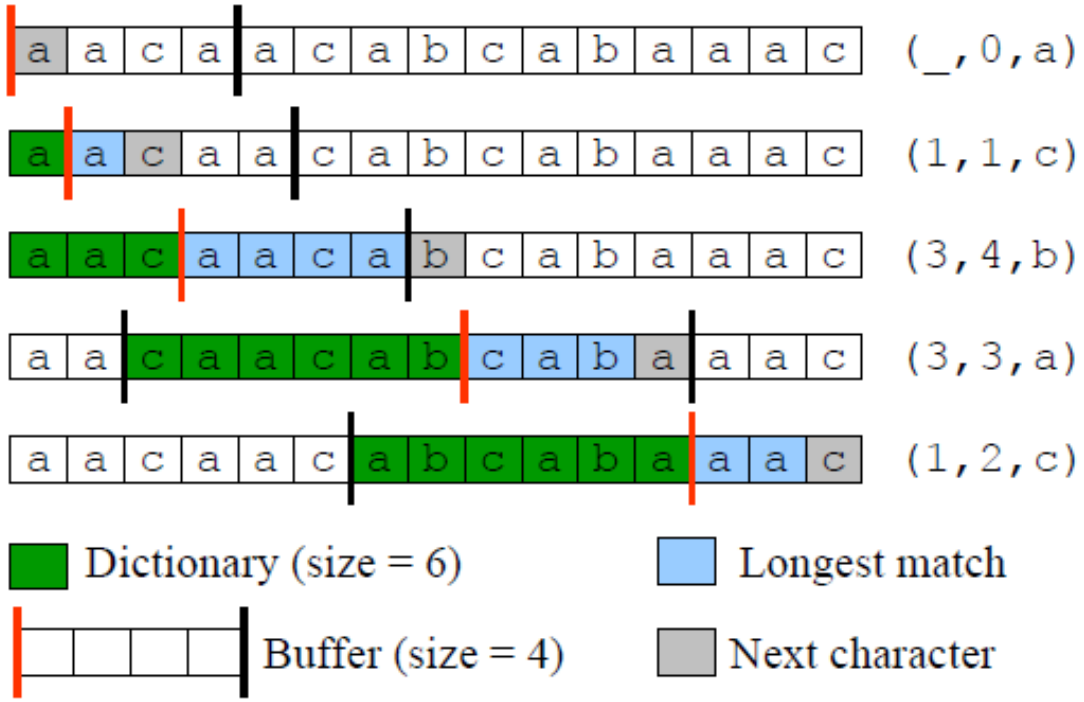
原理：利用数据的**重复结构信息**来进行数据压缩。

**abcabc　　abc-往前看 3 个字，抄 6 个字**

**(距离, 长度, 下一个字符)**

# LZ77算法举例

编码以下数据：aacaacabcabaaac



# LZ77核心概念

滑动窗口：任一时刻，编解码过程仅关注的数据段。

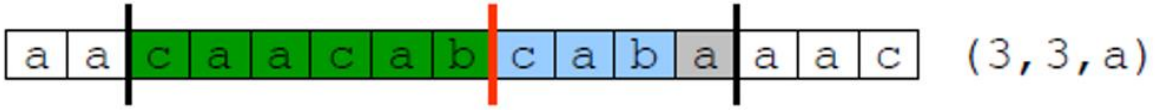
光标：当前需要编码的字符位置，也是字典区域与前向缓冲之间的边界

字典区域：随滑动窗口滑动，用作编码时的参考依据。

前向缓冲：随滑动窗口滑动，采用最大最长匹配算法来匹配字典（包括前向缓冲本身）中出现过的重复数据。

编码格式：在搜索到匹配数据之后（也可以没匹配到），将匹配结果进行编码。

两条黑色线之间：滑动窗口




红线：光标

字典　　前向缓冲

# LZ77算法实现注意事项

如何表示滑动窗口？

- 光标！

# LZ77算法实现注意事项

如何表示滑动窗口？

- 光标！

内存不可能一直朝一个方向无限扩展，如何处理？

1. 移动内容
2. 循环缓冲

# LZ77算法实现注意事项

如何表示滑动窗口？

- 光标！

内存不可能一直朝一个方向无限扩展，如何处理？

1. 移动内容
2. 循环缓冲

哈夫曼编码和LZ77算法各有什么优缺点？

# 参考压缩解压代码

# 实验环节

完成数据备份软件的：

压缩解压部分

（推荐）完成单元测试